

Subject: CS336 Assignment 1, BPE Tokenizer

Date: from 2026/7/20 to 2026/7/21

Contents

BPE Principle and Training	1
-----------------------------------	----------

BPE Principle and Training

Why do we need a Tokenizer?

为什么不能直接把 UTF-8 字节 (0-255) 丢给模型？如果模型直接读取原始字节：

- **计算成本爆炸.** 自注意力 (Self-Attention) 的复杂度是序列长度 N 的平方, 即 $O(N^2)$.
- **例子.** 单词 "Transformer" 占用 11 个字节; 以字节为单位序列长度就是 11, 把它作为一个整体 Token 则长度是 1.
- **结果.** 直接用字节会让输入序列极其漫长, 模型跑得慢, 显存占用也显著增加.

Tokenizer 的本质作用. 在 “词表大小” 和 “序列长度” 之间寻找平衡点, 把常见的字节组合打包成一个数字 (Token), 从而实现对文本的压缩.

BPE (Byte-Pair Encoding). 它是 Tokenizer 的一种主流实现方案: 通过统计学手段自动寻找最值得合并的字节组合.

训练流程. 训练集 $\xrightarrow{\text{UTF-8}}$ 字节序列 (0-255), 然后不断迭代三个动作: 统计相邻字节对 $\langle B_1, B_2 \rangle$ 的频次; 选出频次最大的 best_pair; 将该字节对写入合并字典.

可视化. 分词的中间结果可在 <https://tiktokenizer.vercel.app/> 中查看.

Pre-tokenization

预分词 (pre-tokenization) 是在分词之前先按一组规则把文本切开, 确保模型不会跨越 “分割” 边界去构建 token. 它比 tokenizer 本身更简单, 却决定了统计合并的基本单位.

- **反例.** 若不预分词而直接分词, “您好人没了” 可能被切成 “您”、“好人”、“没了”, 出现与日常经验相悖的词. 先按空格预分词为 “您好”、“人没了”, 再分词得到 “您好”、“人”、“没了”.
- **结构破坏案例.** 语料中的 “It is good”: 错误的 token 是跨词的 ti, 正确的 token 是 It.
- **解决方案.** 用正则把文本切分为 `["It", "is", "good", ".", "!", ""]`, 使 t 只能在单词内部寻找合并对象, 从而保证 token 的完整性.

为什么要预分词? 防止 BPE 学习到跨越语义边界的无意义组合, 让模型更容易学习语义.

BPE Training

BPE 训练并不直接在原始长文本上扫描, 而是通过一个**词频字典**进行高效迭代.

Step 1: 字节化与初始化. 将语料库经 UTF-8 编码转为字节. 初始词表 (vocab) 为 0-255, 每个字节本身即一个独立 Token.

Step 2: 构建统计字典. 统计预分词后每个独立单位 (pre-token) 出现的次数:

$$\{(l, o, w): 5, \quad (l, o, w, e, r): 2, \quad (n, e, w, e, s, t): 6\}$$

每一项都是字节 Token 的元组.

Step 3: 寻找最频繁的字节对. 扫描字典, 累加所有相邻两个 Token 组合出现的总频次:

- (e, w) 出现 6 次 (来自 newest);
- (l, o) 出现 $5 + 2 = 7$ 次 (来自 low 与 lower);
- 胜出者: (l, o) 是当前最频繁的字节对.

Step 4: 合并与记录 (merge). (1) 产生新规则 $(l, o) \rightarrow lo$; (2) 把 lo 加入词表并分配新 ID (如 256); (3) 将字典中所有 (l, o) 替换为 lo:

{(l, o, w): 5, (l, o, w, e, r): 2, ...}

Step 5: 迭代. 重复 Step 3 与 Step 4, 直到词表大小达到预设值 (如 10 000) .

Efficiency: Inverted Index

- 朴素做法. 每合并一个字节对, 就遍历整个字典 (可能包含几万个唯一词) 去替换.
- 优化思路.
 - 建立倒排索引 $\text{indices}[(l, o)] = \{\text{词 1}, \text{词 2}\}$, 记录某个字节对出现在哪些词中;
 - 当确定合并 (l, o) 时, 只修改词 1 与词 2 这两项;
 - 同时更新受影响的邻居: 词 1 原有的相邻关系发生变化, 需要增量地加减 stats 计数.

Special Tokens

定义: Special Tokens 是分词器中具有特定功能、不可拆分的原子单位. 与通过统计学学习到的普通 Token 不同, 它们通常由开发者手动预设, 用于定义序列结构或触发特定行为.

1. 结构化标记: 界定序列边界

- $\langle \text{endoftext} \rangle$ (EOS): 告诉模型该篇文档已经结束. 预训练时通常会把多篇短文拼接在一起, 若没有 EOS 隔离, 模型会误认为前一篇的末尾与后一篇的开头之间存在因果逻辑.
- [PAD] (Padding): 批处理 (Batch) 时对齐不等长的序列, 配合 Attention Mask 告知模型忽略这些占位符.

2. 对话模板: 角色分界

- Chat 模型用特殊 Token 标识 system、user、assistant 的对话起止, 防止模型混淆对话内容与控制指令.

3. 垂类应用: 提速与压缩 在医学、法律等专业领域, 可以把高频长术语手动设为 Special / Added Tokens.

- 案例. 医学术语“系统性红斑狼疮”原本被拆成 [系统, 性, 红, 斑, 狼, 疮] (约 6 个 Token); 设为单一 ID 后, 消耗的 Token 数降为 1 个.
- 目的. 提升推理速度: 前向传播次数直接与生成的 Token 数挂钩, 压缩 Token 数可显著降低推理延迟; 节省上下文窗口: 在有限的上下文长度内能容纳更多专业信息.

Token	类型	作用
$\langle \text{endoftext} \rangle$	EOS	界定文档边界, 避免跨文档的因果混淆
[PAD]	Padding	对齐批内长度, 由 Attention Mask 忽略
system / user / assistant	对话模板	标识角色起止, 区分内容与指令